ARXIV:2405.04434

DeepSeek-V2: Efficient MoE Language Model with Multi-Head Latent Attention

A strong, economical, and efficient Mixture-of-Experts architecture that activates only 21B of 236B parameters

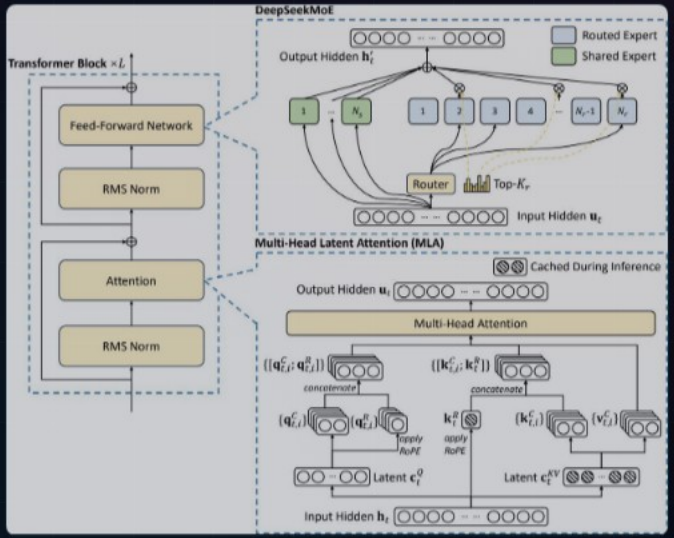
236B	21B	128K
TOTAL PARAMS	ACTIVATED	CONTEXT

The LLM Scaling Dilemma: Performance vs. Cost & Speed

- Larger models yield better intelligence, but training costs grow superlinearly with parameter count.
- Autoregressive generation is bottlenecked by the KV cache: full key-value pairs per head per token consume massive memory and I/O bandwidth.
- Standard compression tricks (GQA, MQA) reduce cache but sacrifice model quality — a frustrating trade-off.
- **The question:** Can we simultaneously cut training costs, shrink KV cache, and preserve (or improve) accuracy?

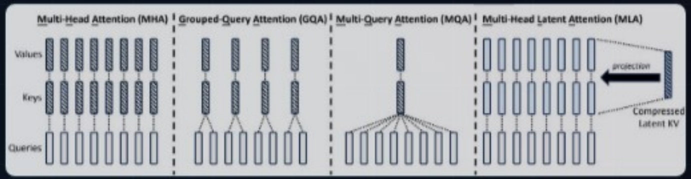
DeepSeek-V2 Architecture: MLA + DeepSeekMoE

- **Multi-Head Latent Attention (MLA)** replaces standard MHA, compressing KV cache via low-rank joint compression.
- **DeepSeekMoE** replaces dense FFN with fine-grained shared + routed experts for sparse activation.
- Both innovations sit inside a standard Transformer block — drop-in replacements with outsized efficiency gains.



Multi-Head Latent Attention:
93.3% KV Cache Compression

- Instead of caching full K and V vectors per head, MLA compresses them into a compact latent vector c^{KV}_t.
- Full keys, values, and a decoupled RoPE-based positional key are reconstructed on-the-fly via learned projection matrices.
- **Result:** Better performance than MHA with dramatically less cache — strictly dominating GQA and MQA.



DeepSeekMoE:
Fine-Grained Experts with Shared Expert Isolation

- **Shared experts (N_s)** are always activated, capturing common knowledge across all tokens.
- **Routed experts (N_r)** are sparsely activated via a top-K_r router, enabling massive parameter counts with low per-token cost.
- **Device-limited routing** constrains each token to experts on at most M devices, minimizing cross-node communication overhead.
- Auxiliary load-balancing losses + token-dropping during training prevent expert overflow without hurting inference.